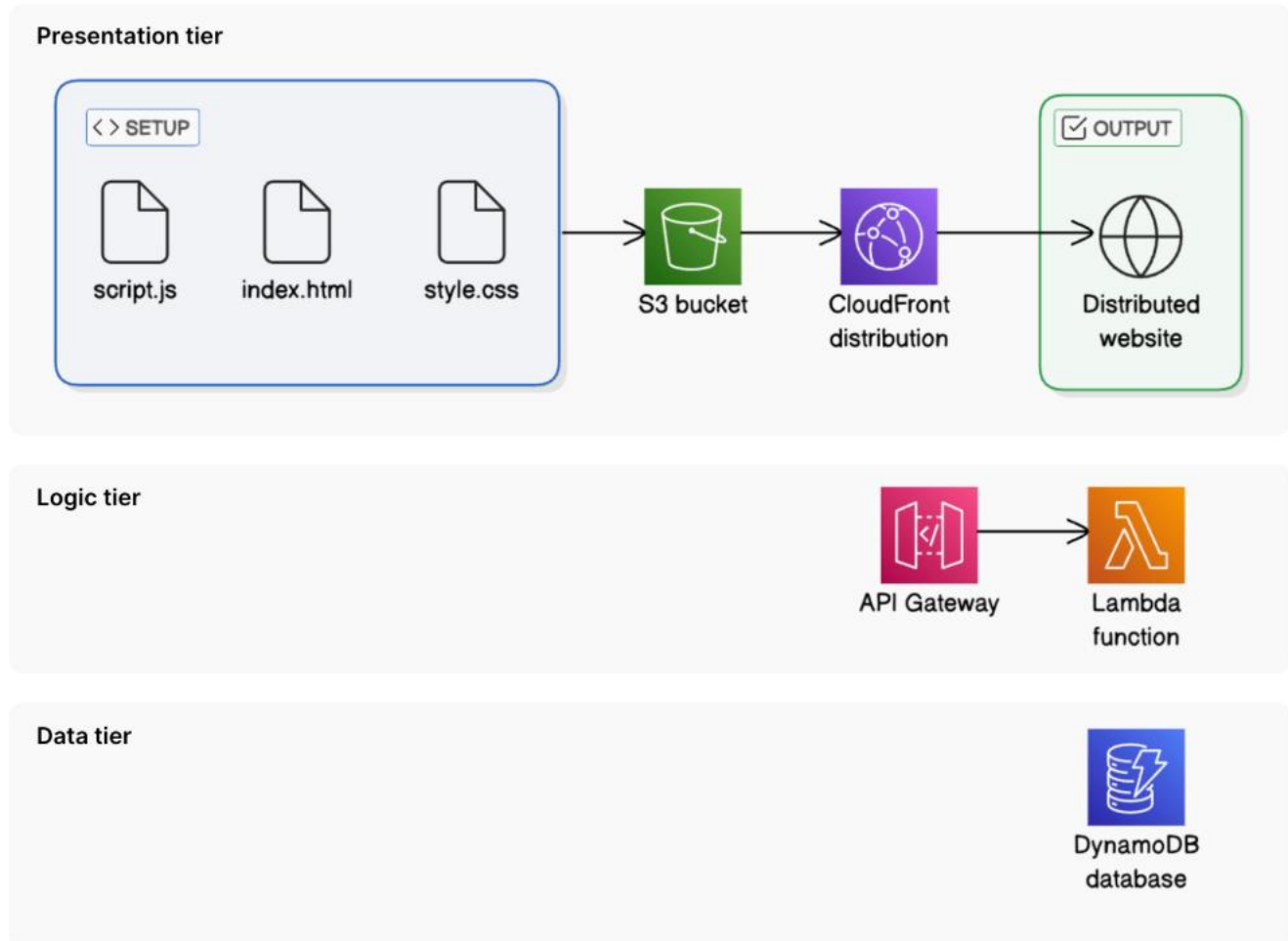


BUILD A THREE-TIER WEB APP - STEP-BY-STEP

DOCUMENTATION



This documentation provides a comprehensive, step-by-step guide to building a three-tier web application on AWS. It integrates Amazon S3 and CloudFront for the presentation tier, AWS Lambda and API Gateway for the logic tier, and Amazon DynamoDB for the data tier. Follow these steps to create, connect, test, and clean up the components.

Step 1: Set Up the Presentation Tier

Create S3 bucket, upload website files, and configure CloudFront distribution for global delivery.

Actions:

- Log in to AWS Management Console and select the region.
- Navigate to S3 -> Create bucket.
- Enter a unique bucket name (e.g., `nextwork-three-tier-<yourname-random>`).
- Leave default settings and create the bucket.
- Upload website files: `index.html`, `style.css`, `script.js`.

- In CloudFront, create a distribution with the S3 bucket as origin.
- Set Origin access to Origin Access Control (create new OAC) and Default root object to index.html.
- Create the distribution and copy the generated policy to update the S3 bucket permissions.

General purpose buckets (1) Info

Buckets are containers for data stored in S3.

Find buckets by name

Name	AWS Region	Creation date
nextwork-three-tier-webapplication	Asia Pacific (Mumbai) ap-south-1	October 2, 2025, 08:20:34 (UTC+05:30)

Account snapshot Info View dashboard

Updated daily

Storage Lens provides visibility into storage usage and activity trends.

External access summary - new Info

Updated daily

External access findings help you identify bucket permissions that allow

nextwork-three-tier-webapplication Info

Objects

Objects (3)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Name	Type	Last modified	Size	Storage class
index.html	html	October 2, 2025, 08:21:09 (UTC+05:30)	564.0 B	Standard
script.js	js	October 2, 2025, 08:21:09 (UTC+05:30)	680.0 B	Standard
style.css	css	October 2, 2025, 08:21:09 (UTC+05:30)	447.0 B	Standard

nextwork-cloudfrontdistribution-three-tier Standard View metrics

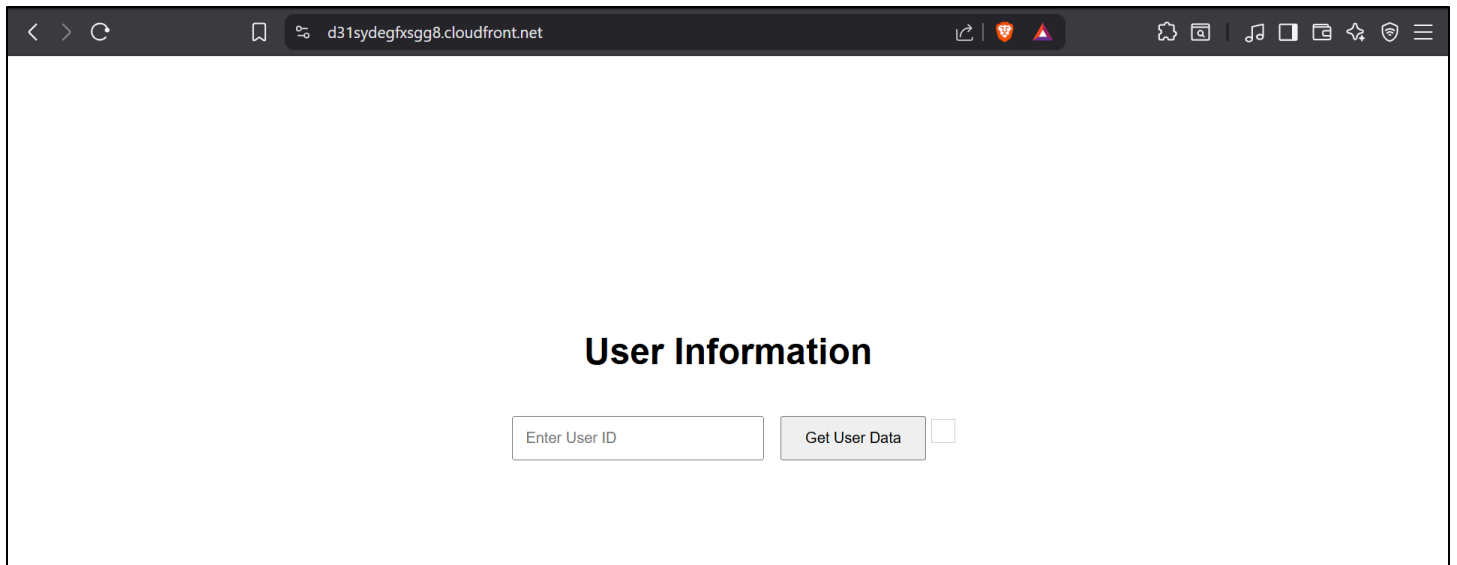
General Security Origins Behaviors Error pages Invalidations Tags Logging

Details

Name nextwork-cloudfrontdistribution-three-tier	Distribution domain name d31sydegfsgg8.cloudfront.net	ARN arn:aws:cloudfront::944362433406:dis-tribution/E31ZUA7R6JM4JP	Last modified October 2, 2025 at 3:24:41 AM UTC
--	--	--	--

Settings Edit

Description - Price class Use all edge locations (best performance) Supported HTTP versions HTTP/2, HTTP/1.1, HTTP/1.0	Alternate domain names - Add domain	Standard logging Off Cookie logging Off Default root object index.html
--	--	--

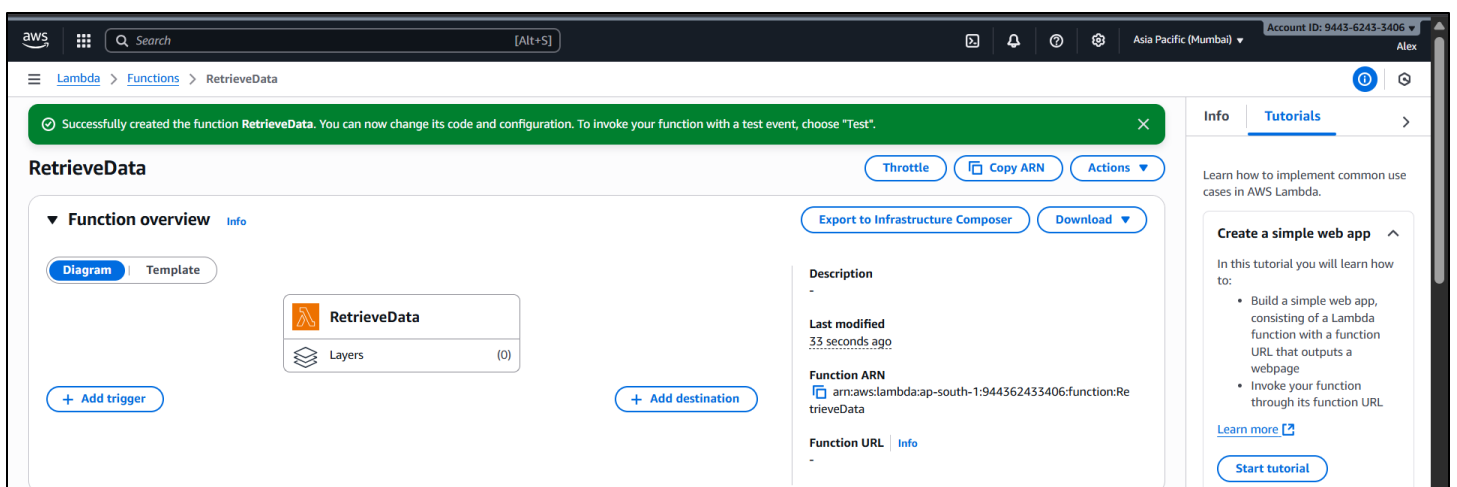


Step 2: Set Up the Logic Tier

Create a Lambda function and an API Gateway REST API to expose the function.

Actions:

- Navigate to Lambda -> Create function -> Author from scratch.
- Enter Function name: RetrieveUserData; Runtime: Node.js; Architecture: x86_64.
- Add the Lambda code that queries DynamoDB (ensure YOUR_REGION is set correctly).
- Deploy the Lambda function.
- In API Gateway, create a new REST API (UserRequestAPI), Endpoint type: Regional.
- Create a resource named /users and add a GET method.
- Integration type: Lambda Function; enable Lambda Proxy Integration and select RetrieveUserData.
- Deploy the API to a new stage named prod and copy the Invoke URL.



Successfully updated the function RetrieveData.

EXPLORER

RETRIEVEDATA
JS index.mjs

DEPLOY
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)

TEST EVENTS (NONE SELECTED)
+ Create new test event

JS index.mjs

```
1 // Import individual components from the DynamoDB client package
2 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
3 import { DynamoDBDocumentClient, GetCommand } from "@aws-sdk/lib-dynamodb";
4
5 const ddbClient = new DynamoDBClient({ region: 'ap-south-1' });
6 const ddb = DynamoDBDocumentClient.from(ddbClient);
7
8 async function handler(event) {
9   const userId = event.queryStringParameters.userId;
10   const params = {
11     TableName: 'UserData',
12     Key: { userId }
13   };
14
15   try {
16     const command = new GetCommand(params);
17     const { Item } = await ddb.send(command);
18     if (Item) {
19       return {
20         statusCode: 200,
21         body: JSON.stringify(Item),
22         headers: { 'Content-Type': 'application/json' }
23       };
24     } else {
25       return {
26         statusCode: 404,
27         body: JSON.stringify({ message: "No user data found" }),
28         headers: { 'Content-Type': 'application/json' }
29       };
30     }
31   } catch (error) {
32     console.error(error);
33     return {
34       statusCode: 500,
35       body: JSON.stringify({ message: "Internal server error" }),
36       headers: { 'Content-Type': 'application/json' }
37     };
38   }
39 }
```

Info

Tutorials

Learn how to implement common use cases in AWS Lambda.

Create a simple web app

In this tutorial you will learn how to:

- Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
- Invoke your function through its function URL

Learn more

Start tutorial

CloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search [Alt+S]

Asia Pacific (Mumbai)

Account ID: 9443-6243-3406

Alex

API Gateway

APIs

Resources - UserRequestAPI (v323tvvzdb)

API Gateway

APIs

Custom domain names

Domain name access associations

VPC links

API: UserRequestAPI

Resources

Stages

Authorizers

Gateway responses

Models

Resource policy

Documentation

Dashboard

API settings

Resources

Create resource

/

/users

GET

/users - GET - Method execution

ARN: arn:aws:execute-api:ap-south-1:944362433406:v323tvvzdb/*/GET/users

Resource ID: tgc8jj

API actions

Deploy API

Update documentation

Delete

Client

Method request

Integration request

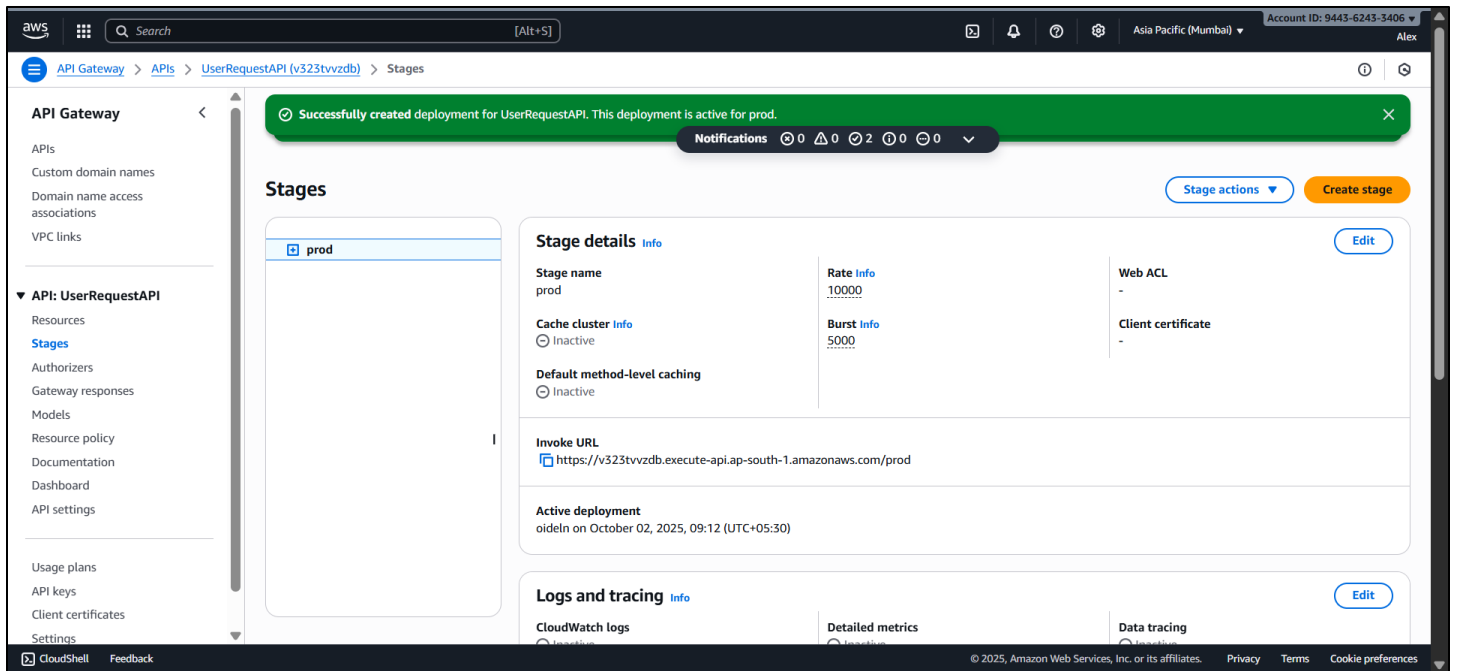
Method response

Integration response

Proxy integration

Method request settings

Edit



Step 3: Set Up the Data Tier

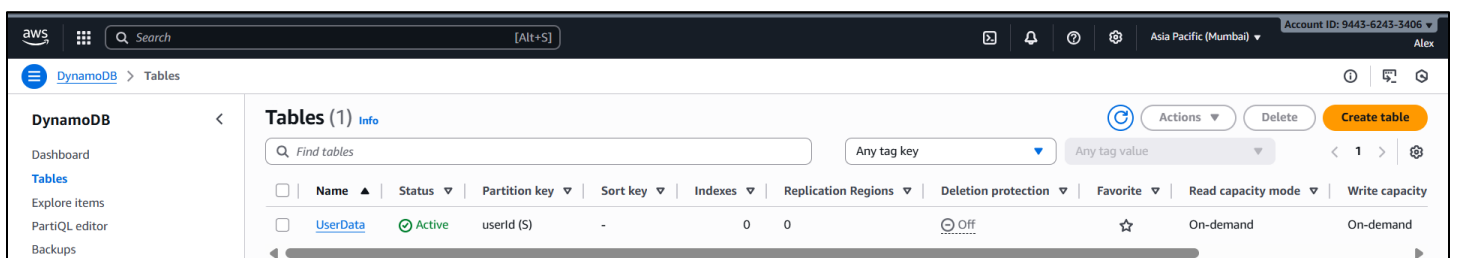
Create a DynamoDB table and add a sample item that the Lambda function can retrieve.

Actions:

- Navigate to DynamoDB -> Create table.
- Table name: UserData. Partition key: userId (String). Keep defaults and create the table.
- Wait for the table status to become Active.
- Open the table -> Explore table items -> Create item -> Switch to JSON view.
- Paste and create the sample item:

```
{
  "userId": "1",
  "name": "Test User",
  "email": "test@example.com"
}
```

- Verify the item appears in the table.



▼ DAX

Clusters

Subnet groups

Parameter groups

Events

Table: UserData - Items returned (2)

Scan started on October 02, 2025, 09:21:21

Actions ▼

Create item

< 1 >

⚙

☐	userid (String) ▼	email ▼	name ▼
☐	2	test2@exa...	Test User 2
☐	1	test@exam...	Test User

Step 4: Integrate the Tiers

Connect the frontend to the API, ensure CORS is configured, and update script.js with the API URL.

Actions:

- Update script.js to call the API Invoke URL (append /users?userId=1 when testing).
- Upload the updated script.js to the S3 bucket and invalidate CloudFront cache if necessary.
- Test the API Invoke URL directly to ensure it returns the DynamoDB item.
- If the frontend shows errors, open browser developer tools (F12) and inspect the Console/Network tabs.
- Enable CORS on API Gateway for the /users resource and include GET and OPTIONS methods.
- Redeploy the API to the prod stage.
- Update Lambda responses to include CORS headers (Access-Control-Allow-Origin) when using Lambda Proxy Integration.



< > ↺ v323twvzdb.execute-api.ap-south-1.amazonaws.com/prod/users?userId=1

```

{
  "email": "test@example.com",
  "name": "Test User",
  "userId": "1"
}
  
```

Raw Parsed

The screenshot displays a web browser window with the address bar showing `d31sydegfxsgg8.cloudfront.net`. The page content is titled "User Information" and features a text input field containing the number "1" and a button labeled "Get User Data".

The browser's developer console is open, revealing four error messages:

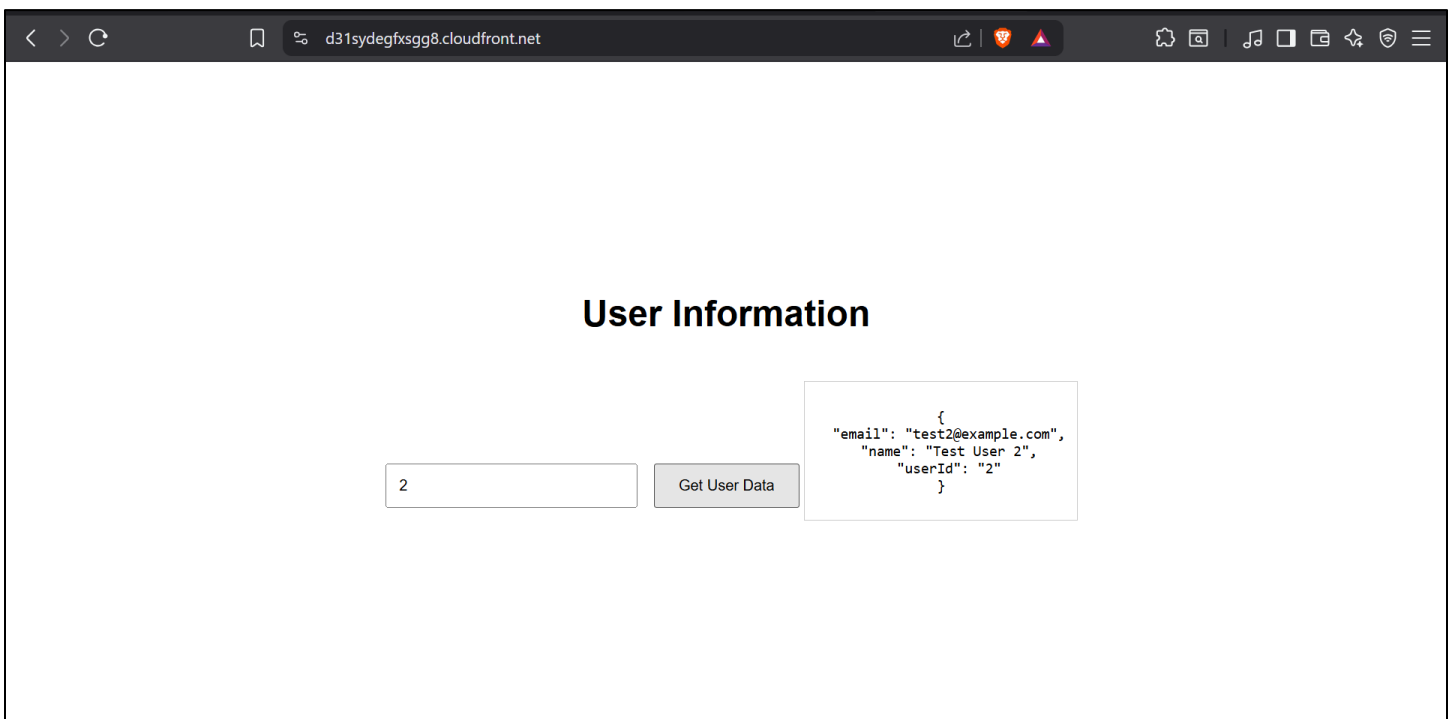
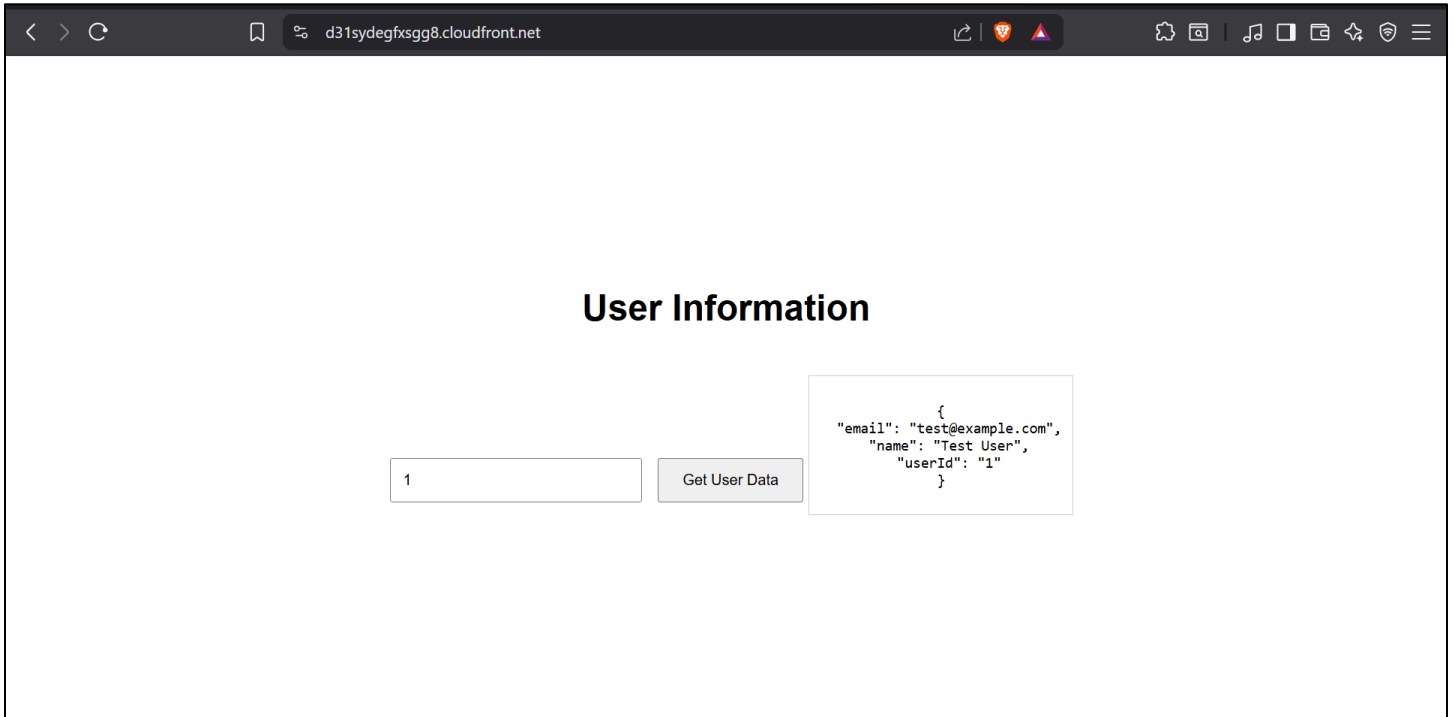
- Failed to load resource: the server responded with a status of 403 ()**: This error occurs while attempting to load the `favicon.ico` file.
- Access to fetch at 'https://v323tvvzdb.execute-api.ap-south-1.amazonaws.com/prod/users?userId=1' from origin 'https://d31sydegfxsgg8.cloudfront.net' has been blocked by CORS policy: No 'Access-Control-Allow-Origin' header is present on the requested resource.**: This is a Cross-Origin Resource Sharing (CORS) error preventing the browser from fetching user data from the specified API endpoint.
- Failed to load resource: v323tvvzdb.execute-a_od/users?userId=1 net::ERR_FAILED**: This error indicates a failure to load a resource, likely related to the user data endpoint.
- Failed to fetch user data: TypeError: Failed to fetch at fetchUserData (script.js:9:32) at HTMLButtonElement.onclick ((index):13:43)**: This is a JavaScript runtime error occurring in `script.js` at line 9, column 32, triggered by the button click event handler.

Step 5: Validate a Fully Functioning Web App

Final checks to confirm the three-tier app works end-to-end and common troubleshooting steps.

Actions:

- Access the CloudFront domain and use the UI to request user data (userId = 1).
- If no data appears, check script.js for the correct API URL and re-upload.
- Use browser devtools to identify CORS or network errors; fix by enabling CORS and updating Lambda headers.
- Inspect CloudWatch logs for Lambda to debug backend errors.



Conclusion

Congratulations! You have built a complete three-tier web application using S3, CloudFront, Lambda, API Gateway, and DynamoDB. This architecture separates concerns across presentation, logic, and data tiers, making your application scalable, maintainable, and secure.